Project Friday

Architecture & Design Document

A self-improving, tool-using AI agent
with a ReAct brain, sandboxed OS hands,
and a growing skill library.

Version 0.2.0

April 23, 2026

Stack: Python 3.11+ · FastAPI · LangGraph · LangChain · Next.js 15 · React 19 · TypeScript

Contents

1 Vision & Scope **2**
1.1 Design Philosophy . . . 2

2 System Architecture **2**
2.1 Component Responsibilities . . . 2
2.2 LangGraph Node/Edge Visualization . . . 3

3 Architecture Decisions (Why) **3**

4 Agent State **3**

5 Tools **3**
5.1 File Operations . . . 3
5.2 Shell Execution . . . 3
5.3 Web Search . . . 5
5.4 Market Data . . . 5
5.5 Skill Library . . . 5
5.6 Skill Agents . . . 5
5.7 Dynamic MCP Tools . . . 5

6 Skill Agent System **5**
6.1 Directory Structure . . . 5
6.2 Manifest Schema . . . 6
6.3 Lifecycle . . . 6

7 Security Model **7**
7.1 Path Sandboxing . . . 7
7.2 Command Allowlist . . . 7
7.3 Skill Script Execution . . . 7
7.4 Retry & Recursion Limits . . . 8

8 System Prompt **8**

9 LLM Provider Configuration **8**

10 Frontend Architecture **8**
10.1 Component Tree . . . 8
10.2 SSE Stream Hook . . . 9
10.3 Event Types . . . 9

11 Project Structure **9**

12 Future Enhancements **10**

1 Vision & Scope

Friday is a personal agentic AI assistant that reasons over a set of tools to complete tasks—from fetching recent CVEs to scaffolding a Next.js component. It operates on a **ReAct loop** (Reason → Act → Observe → Reason again) rather than a linear pipeline, meaning it genuinely decides what to do next instead of following a hardcoded flow.

1.1 Design Philosophy

1. **Agent, not chatbot.** Every response is the result of tool-augmented reasoning.
2. **Self-improving.** The Voyager-inspired skill library lets Friday write, test, and commit reusable scripts and framework agents.
3. **Security by default.** All OS access is sandboxed; shell commands are allowlisted; external directories require explicit opt-in.
4. **Local-first.** LM Studio is the default provider—no cloud API keys required to start.

2 System Architecture

```
+-------------------------------------------------------+
|                  Next.js Frontend                      |
|  Chat UI --SSE Stream--> /api/friday (route handler)   |
+----------------------------+---------------------------+
                             | HTTP POST
+----------------------------v---------------------------+
|                  FastAPI Backend                       |
|  POST /chat --> LangGraph executor --SSE--> client     |
|  GET  /workspace --> file tree snapshot                |
|  GET  /skills    --> registered skill index            |
|  GET  /agents    --> registered Skill Agents           |
+----------------------------+---------------------------+
                             |
+----------------------------v---------------------------+
|               LangGraph State Machine                  |
|                                                        |
|   +----------+  tool_call?  +--------------------+     |
|   |  Agent   | --- YES ---> |  Tool Executor     |     |
|   |  Node    | <----------- |  Node (cached)     |     |
|   |  (LLM)  |              +--------------------+     |
|   +----+-----+                                         |
|        | NO (final answer)                             |
|        v                                               |
|      END                                               |
+-------------------------------------------------------+
```

High-level data flow

2.1 Component Responsibilities

Component	Responsibility
`main.py`	FastAPI application, routes, SSE streaming, CORS
`agent/graph.py`	LangGraph `StateGraph` definition, cached tool node
`agent/nodes.py`	Agent node (LLM invocation with system prompt), router, retry logic
`agent/state.py`	`AgentState` TypedDict flowing through every node
`agent/system_prompt.py`	Friday's identity, ReAct instructions, dynamic context injection
`agent/model.py`	Multi-provider LLM factory (LM Studio, Groq, Ollama)
`agent/tools/`	All tool implementations (see Section 5)

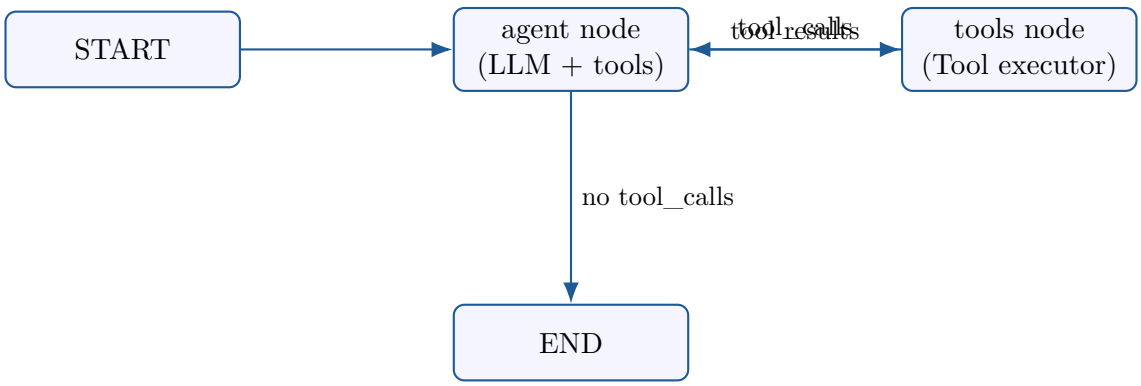

Figure 1: Friday's LangGraph control flow (nodes and directed edges).

2.2 LangGraph Node/Edge Visualization

Note

The graph is intentionally cyclic (agent ↔ tools). This is what enables iterative ReAct behavior: the LLM can inspect tool output and decide the next action until completion.

3 Architecture Decisions (Why)

4 Agent State

The `AgentState` TypedDict flows through every node:

5 Tools

5.1 File Operations

5.2 Shell Execution

Note

Shell commands are validated against a configurable **allowlist** (first token must match). Shell chaining operators (`&&`, `||`, `;`, `|`) are rejected. Timeout defaults to 120 seconds.

Decision	Why This Choice	Trade-off Accepted
LangGraph ReAct loop	Needed explicit cyclic control flow for tool use and retries instead of one-shot prompting.	More state management complexity than a linear chain.
FastAPI + SSE	Streaming intermediate thought/tool events improves observability and UX responsiveness.	SSE parsing/dedup logic is more complex than plain JSON responses.
Sandbox + allowlist shell policy	Reduces blast radius for command execution and path traversal risks.	Some legitimate commands require explicit allowlist updates.
Skill library + dynamic tool loading	Makes capabilities persistent and reusable across sessions; enables self-improvement.	Requires index management and tool-name collision handling.
Skill Agents (framework manifests)	Encodes framework-specific best practices so outputs stay consistent over time.	Adds maintenance overhead for guide quality/versioning.
Dynamic MCP registry tools	Lets Friday learn external API-style tools at runtime instead of hardcoding all integrations.	Runtime registration can fail if endpoint metadata is poor.
Prompt/context budgeting	Prevents local-model failures on small context windows (e.g., 4k) by trimming prompt/history.	Some long-tail context can be dropped on very long chats.
Local-first LM Studio default	Fast local iteration without cloud spend/rate limits.	Smaller local models may need stricter prompt discipline.

Field	Type	Purpose
`messages`	`list`	Conversation history (LangGraph's `add_messages` reducer)
`tool_attempts`	`int`	Retry counter for the current tool/script
`active_skill`	`str|None`	Path of the skill script being executed
`skill_context`	`str|None`	Loaded Skill Agent style guide (injected into system prompt)
`target_directory`	`str|None`	External project directory (must be in `ALLOWED_TARGET_DIRS`)
`error_history`	`list[str]`	Rolling log of error messages for retry decisions

Tool	Description
`list_files`	Lists files/folders at a given workspace-relative path
`read_file`	Reads full contents of a workspace file
`write_to_file`	Creates or overwrites a workspace file
`append_to_file`	Appends content to an existing file (or creates it)
`create_directory`	Creates a directory (and parents) inside the workspace
`delete_file`	Deletes a single file (not directories) from the workspace

Tool	Description
`execute_bash_command`	Runs an allowlisted command inside the workspace sandbox
`execute_in_directory`	Runs an allowlisted command in an external allowed directory

5.3 Web Search

`web_search` queries the web via Tavily (if configured) or DuckDuckGo (fallback), returning the top 3 result summaries.

5.4 Market Data

Tool	Description
`get_stock_quote`	Fetches quote data for a ticker using Yahoo Finance (price, change, market cap, P/E where available)
`compare_stock_prices`	Fetches and compares two ticker symbols in a single call

5.5 Skill Library

Tool	Description
`save_to_skill_library`	Moves a tested script from workspace into the skills directory
`search_skill_library`	Finds the closest matching skill by description similarity

5.6 Skill Agents

Tool	Description
`create_skill_agent`	Creates a new framework-specific Skill Agent with manifest, style guide, and templates directory
`load_skill_context`	Loads an agent's manifest and style guide into the LLM context
`list_skill_agents`	Lists all registered Skill Agents and their descriptions
`update_skill_agent`	Patches an existing agent's rules, description, or scaffold steps

5.7 Dynamic MCP Tools

6 Skill Agent System

The Skill Agent system is the core differentiator—it allows Friday to **teach itself** new frameworks and domains at runtime.

6.1 Directory Structure

Tool	Description
`register_mcp_tool`	Registers an external endpoint at runtime; creates dynamic tool `mcp_<name>`
`list_registered_mcp_tools`	Lists currently registered MCP tools from `skills/mcp_tools.json`
`unregister_mcp_tool`	Removes a runtime-registered MCP tool
`mcp_call`	Directly invokes a registered MCP tool by server/name

```
skills/
  index.json                # Script skill registry
  agents/
    nextjs/
      manifest.json         # Metadata, triggers, dos/don'ts
      style_guide.md        # Auto-generated best-practice doc
      templates/            # Optional starter files
    fastapi/
      manifest.json
      style_guide.md
      templates/
```

Skill Agent layout

6.2 Manifest Schema

Each Skill Agent is defined by a `manifest.json` with the following fields:

Field	Type	Description
`name`	string	Slugified agent name (e.g. `nextjs`)
`display_name`	string	Human-readable name (e.g. `Next.js`)
`version`	string	Semantic version
`description`	string	What this agent handles
`trigger_patterns`	string[]	Keywords that activate this agent
`rules.do`	string[]	Best-practice rules to follow
`rules.dont`	string[]	Anti-patterns to avoid
`scaffold_steps`	string[]	Shell commands to set up a new project
`created_at`	ISO 8601	Creation timestamp
`updated_at`	ISO 8601	Last modification timestamp
`auto_generated`	boolean	Whether the LLM created this agent

6.3 Lifecycle

1. **Detection:** User asks to work with a framework (e.g. "Create a Next.js project").
2. **Lookup:** Agent calls `search_skill_library("next.js")`.
3. **Creation (first time):** No match found → agent calls `create_skill_agent(...)` with framework-specific dos, don'ts, scaffold steps, and trigger patterns.
4. **Loading:** Agent calls `load_skill_context("nextjs")` → style guide is injected into the system prompt.

5. **Execution:** Agent follows the loaded rules strictly while scaffolding the project.
6. **Evolution:** If the agent learns new patterns or encounters errors, it calls `update_skill_agent(...)` to refine the rules.

Tip

On subsequent requests involving the same framework, the agent skips creation and immediately loads the existing agent's context—making it faster and more consistent over time.

7 Security Model

7.1 Path Sandboxing

All file operations go through `safe_path()`, which enforces:

- **No absolute paths** inside the workspace.
- **No traversal** via `../../` or similar.
- **Symlink detection:** if resolving a path follows a symlink, the target must still be inside the workspace.
- **Blocked segments:** paths containing `.env`, `.ssh`, `.git`, `.gnupg`, or `.aws` are rejected.

For operations outside the workspace, `safe_target_path()` validates against `ALLOWED_TARGET_DIRS`.

7.2 Command Allowlist

The blocklist-based approach was replaced with an **allowlist**:

- Only the first token of a command is checked against a configurable list of permitted prefixes.
- Shell chaining operators (`&&`, `||`, `;`, `|`, backticks, `$()`) are rejected.
- The allowlist is stored in the `COMMAND_ALLOWLIST` environment variable.

Warning

The command is still executed with `shell=True` for compatibility with complex commands. For maximum security in production, consider switching to container-based isolation (Docker, Firejail, or Windows Sandbox).

7.3 Skill Script Execution

Committed skill scripts are executed with `subprocess.run([sys.executable, path, ...args])`—**no shell injection** is possible because:

- Arguments are passed as a list, not interpolated into a string.
- `shell=True` is NOT used for skill execution.
- The Python interpreter is resolved via `sys.executable`.

Limit	Default	Purpose
`MAX_TOOL_ATTEMPTS`	3	Max retries for a failing tool before bailing
`RECURSION_LIMIT`	25	Hard cap on LangGraph cycles per request
`MAX_TOKENS`	2048	Max tokens per LLM response
`COMMAND_TIMEOUT`	120s	Max runtime for any subprocess

7.4 Retry & Recursion Limits

8 System Prompt

The agent receives a comprehensive system prompt on every invocation, assembled by `build_system_prompt()`:

1. **Static core:** Identity, ReAct instructions, tool usage rules, skill agent workflow, self-improvement protocol, and safety constraints.
2. **Dynamic skill context:** When a Skill Agent is loaded, its style guide (dos, don'ts, scaffold steps) is appended as an additional section.
3. **Extra instructions:** Per-task overrides can be injected programmatically.

This ensures the LLM always knows:

- It is called "Friday" and operates as an agent, not a chatbot.
- Which tools are available and when to use each.
- How to find, create, and follow Skill Agents.
- Its safety boundaries.

9 LLM Provider Configuration

Provider	Model	Use Case
LM Studio (local)	auto-detected	Default—fast, free, offline
Groq	`llama-3.3-70b-versatile`	Speed-optimized cloud fallback
Ollama	`qwen2.5-coder:7b`	Offline code generation

The provider is selected via `MODEL_PROVIDER` in `.env`. LM Studio uses auto-discovery (`/v1/models` endpoint) to find loaded models.

10 Frontend Architecture

10.1 Component Tree

```
RootLayout (layout.tsx)
  HomePage (page.tsx)
    FridayChat
      MessageBubble (per event)
      ThinkingIndicator (during streaming)
      WorkspacePanel | SkillsPanel (sidebar tabs)
```

React component hierarchy

10.2 SSE Stream Hook

`useFridayStream()` manages the full lifecycle:

1. Creates a conversation ID (persists across messages in one session).
2. Sends the query via `POST /api/friday`.
3. Reads the SSE stream via `ReadableStream` API.
4. Deduplicates events to prevent repeated rendering.
5. Supports abort (Stop button) and reset (New Chat).

10.3 Event Types

Type	Source	UI Rendering
`user`	Frontend	Right-aligned blue bubble
`thought`	Agent reasoning	Dimmed dashed-border card with "Reasoning step" badge
`tool_call`	Agent → tool	Dashed card with "Tool call" badge and JSON args
`tool_result`	Tool → agent	Collapsible `<details>` with "Tool result" badge
`final`	Agent answer	Full left-aligned message bubble

11 Project Structure

```
Project-Friday/
  README.md
  Project-info.txt
  run-friday.ps1                # One-command launcher
  docs/
    friday_architecture.tex     # This document
    friday_api_reference.tex    # API reference
  friday-backend/
    main.py                     # FastAPI app + routes
    requirements.txt
    .env / .env.example
    agent/
      __init__.py
      graph.py                  # LangGraph state machine
      nodes.py                  # Agent node, router, retry
      state.py                  # AgentState TypedDict
      system_prompt.py          # System prompt builder
      model.py                  # LLM provider factory
      tools/
        __init__.py             # Tool registry
        os_tools.py             # File + shell tools
        web_search.py           # Tavily / DuckDuckGo
        market_data.py          # Yahoo Finance quote/compare tools
        mcp_registry.py         # Runtime MCP registration + dynamic tools
        mcp_client.py           # MCP invocation wrapper
        skill_library.py        # Save / search skills
        skill_agent.py          # Skill Agent CRUD
    workspace/                  # Sandboxed file system
```

```
    skills/
      index.json                # Script skill registry
      mcp_tools.json            # Runtime MCP tool registry
      agents/                   # Skill Agent definitions
  friday-frontend/
    app/
      page.tsx                  # Root page
      layout.tsx                # Fonts + metadata
      globals.css               # Design tokens
      api/friday/
        route.ts                # Proxy: chat + workspace
        skills/route.ts         # Proxy: skills
        agents/route.ts         # Proxy: agents
    components/
      FridayChat.tsx            # Main chat container
      MessageBubble.tsx         # Event renderer
      ThinkingIndicator.tsx     # Streaming dots
      WorkspacePanel.tsx        # File tree sidebar
      SkillsPanel.tsx           # Skills + agents sidebar
    hooks/
      useFridayStream.ts        # SSE consumer hook
    lib/
      api.ts                    # Typed API client
```

Full monorepo layout

12 Future Enhancements

1. **Semantic skill search:** Replace `SequenceMatcher` with embedding-based lookup (ChromaDB or LanceDB).
2. **Persistent memory:** Replace `MemorySaver` with SQLite or file-based checkpointer for conversation persistence across restarts.
3. **Richer MCP transport:** Add full protocol-level MCP transport/session support beyond simple HTTP endpoint wrappers.
4. **Multi-language skills:** Support non-Python skill scripts (Node.js, Bash, Go).
5. **Container isolation:** Run tool commands in Docker containers for production-grade sandboxing.
6. **Authentication:** Add API key or JWT-based auth for production deployments.